
ARCHITECTURE BLUEPRINT · MARKETING SUBSYSTEM DEEP
DIVE

Necoyoad

v4

*A focused deep-dive into the email-marketing pipeline:
newsletters, campaigns, tracking, and the cron sender*

`scope:` 41 files, ~17,000 LOC (6 controllers, 5 models,
7 cron tasks, 9 email libs, 10 API endpoints)
`trace:` 6-stage lifecycle (compose → commit → send → open → click)
`findings:` 26 cross-cutting (vestigial bounce, double link rewrite,
broken API PUTs, no unsubscribe, PHPMailer 5.0)
`listings:` 18 PHP · `pages:` 27

Reverse-Engineered Architecture Report

Verified against PHP source commit `ce75adf`
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — Marketing & Campaign Subsystem (v4)

*A focused deep-dive into the email-marketing pipeline:
newsletters, campaigns, contacts, tracking, and the cron sender*

Abstract

This is the fourth volume of the Necoyoad architecture blueprint. v1 reconstructed the platform from the SQL dump. v2 verified v1 against the source. v3 deep-dived the presentation layer. v4 zooms in on a single subsystem: the *marketing/campaign pipeline* — how a newsletter becomes a campaign, how the campaign becomes per-recipient emails, how those emails are sent via SMTP, how opens and clicks are tracked, and how bounces are (not) processed.

The marketing subsystem is a self-hosted Mailchimp clone embedded in an e-commerce platform. It comprises: (i) a newsletter editor with a WYSIWYG HTML body and `{%placeholder%}` personalisation tokens; (ii) a campaign composer that targets contact lists, schedules the send, scores the body against spam rules, and rewrites links into trackable URLs; (iii) a per-recipient task queue (`task_queue`) driven by a CLI cron that sends up to 50 emails per pass with a 15-minute defer; (iv) a tracking pixel endpoint (`marketing/campaign/trace`) that writes open events to `campaign_stat`; (v) a link-redirect endpoint (`marketing/campaign/link`) that writes click events to `campaign_link_stat` and 302-redirects to the original URL; (vi) a PHPMailer 5.0 (2009) SMTP workhorse renamed **Mailer**; (vii) a vestigial bounce processor that is commented out of the cron and references non-existent Interspire SendStudio classes; and (viii) auxiliary birthday/promoter/seller email tasks.

This paper documents all eight components, with 18 verbatim PHP code listings, 6 TikZ diagrams, a full 6-stage campaign lifecycle trace (newsletter create → campaign compose → campaign commit → cron send → email open → email click), the complete marketing file inventory (41 files, ~17,000 LOC), and 26 cross-cutting findings. The stance is unchanged: **descriptive, not prescriptive**. No improvements are proposed.

Keywords: Necoyoad, OpenCart lineage, PHP 8.1.2, email marketing, campaign pipeline, newsletter, contact list, NecoMailer, PHPMailer 5.0, SMTP, POP3, bounce processing, tracking pixel, link rewriting, `task_queue`, cron, EAV property table, spam rules, Venezuelan locale.

Contents

1	Introduction and Scope	4
1.1	Four Volumes, Four Scales	4
1.2	Why the Marketing Subsystem?	4
1.3	Methodology	4
1.4	Document Roadmap	5
2	Thirty-Second Overview	5
3	The Marketing Database Schema	5
3.1	The 9 Marketing Tables	6
3.2	Notable Schema Details	6
3.2.1	The <code>campaign_contact.task_queue_id</code> Column	6
3.2.2	The <code>campaign_link</code> Dual-Column URL Storage	6
3.2.3	The Telemetry Tables	7
3.2.4	The Mailserver Storage Overload	7
4	Newsletter Composition	7
4.1	The Newsletter Controller	7
4.2	The Personalisation Tokens	8
4.3	The Model Schema Mismatch	8
5	Contacts and Contact Lists	9
5.1	The Contact Controller	9
5.1.1	The Import Wizard	9
5.1.2	The vCard Export	9
5.2	The Contact List Controller	9
5.3	The <code>ModelMarketingList</code> Type Bug	9
6	The Campaign Composer and Commit	10
6.1	Phase 1 — Compose (<code>marketing/campaign/insert</code>)	10
6.2	Phase 2 — Commit (<code>marketing/campaign/send</code>)	10
6.3	The Double Link Rewriting	11
7	The Cron Sender	12
7.1	The <code>sendCampaign</code> Algorithm	12
7.2	The Locking Mechanism	14
7.3	The 50-Email Throttle	14
7.4	The Per-Pass Cache	14
8	The Tracking Endpoints	14
8.1	The Tracking Pixel (<code>marketing/campaign/trace</code>)	15
8.2	The Link Redirect (<code>marketing/campaign/link</code>)	15
8.3	The <code>trackEmail</code> SQL Bug	16
9	The Mailer (PHPMailer 5.0)	16
9.1	Two Parallel Mailers	16
9.2	The SMTP Client	16
9.3	The POP3 Client (for Bounce)	17
9.4	The Password Storage	17
10	The Vestigial Bounce Processor	17

11 The Auxiliary Tasks: Birthday, Promoter, Seller	17
11.1 CronBirthday (system/cron/api/birthday.php, 97 lines)	17
11.2 CronPromoter (system/cron/api/promoter.php, 167 lines)	18
11.3 CronSeller (system/cron/api/seller.php, 92 lines)	18
11.4 CronUpdate (system/cron/api/update.php, 214 lines)	18
11.5 The Hidden link2Campaign Mechanism	18
12 The Spam Rules	18
12.1 The Three-Layer Spam Pipeline	18
12.2 The Rule Format	19
13 The Admin API Marketing Endpoints	19
13.1 The 5 Endpoints	19
13.2 The Broken PUT Branches	19
14 The Campaign Lifecycle Trace	19
14.1 Stage 1 — Newsletter Create	19
14.2 Stage 2 — Campaign Compose	20
14.3 Stage 3 — Campaign Commit	20
14.4 Stage 4 — Cron Send	21
14.5 Stage 5 — Email Open (Tracking Pixel)	22
14.6 Stage 6 — Email Click (Link Redirect)	22
14.7 Lifecycle Summary	22
15 Cross-Cutting Findings	22
15.1 Architecture	22
15.2 Bugs and Vestigial Code	23
15.3 Missing Features	23
15.4 Design Choices	23
16 Closing Observations	24
16.1 The Marketing Subsystem is a Self-Hosted Mailchimp Clone	24
16.2 The Subsystem is Where the Task Scheduler Does Real Work	24
16.3 The Subsystem is Where v1's Inferences are Most Incomplete	24
16.4 The Subsystem is Remarkably Complete Despite Its Warts	24
16.5 Final Note	24
A Marketing File Inventory	25
B Glossary	25

1 Introduction and Scope

1.1 Four Volumes, Four Scales

This is the fourth volume of the Necoyoad architecture blueprint. The four volumes operate at different scales and on different subsystems:

- **v1 — SQL-only reconstruction.** 35 pages. The whole platform from the 87-table dump.
- **v2 — Source verification.** 38 pages. v1 verified against the source, plus the bootstrap, engine, dual Hooks/Events, password algorithm, order flow, cron CLI, admin REST API, mobile strategy (with widget scoping).
- **v3 — Presentation layer deep dive.** 37 pages. The theme/widget rendering pipeline, the composite-view pattern, the LIKE '%key=value%' widget query pattern, the visual theme editor.
- **v4 — This volume.** The marketing/campaign subsystem. How a newsletter becomes a sent, tracked, personalised email.

1.2 Why the Marketing Subsystem?

The marketing subsystem is the second most intricate part of Necoyoad (after the presentation layer covered in v3). It is a self-hosted Mailchimp clone — newsletter composition, contact-list targeting, scheduled sending, open/click tracking, bounce handling — embedded in an e-commerce platform. It is also where the `task_queue` scheduler (introduced in v2) does its most consequential work: a campaign of 1,000 recipients is literally 1,000 queued tasks, each sending one personalised email.

The subsystem is also notable for what it *doesn't* do. v4 documents several vestigial or broken features the source reveals:

- The bounce processor is commented out of the cron and references non-existent Interspire SendStudio classes.
- There is no unsubscribe mechanism — no `?unsubscribe=1` endpoint, no unsubscribe header.
- There is no forward-to-a-friend endpoint (a hidden `link2Campaign` mechanism could enable it but no admin UI exposes it).
- All five admin API v1.0.0 marketing PUT branches are broken in different ways.
- The `campaign_contact.task_queue_id` schema column is declared but never written.
- Per-recipient link rewriting happens *twice* per campaign (admin commit phase + cron send phase), making the admin's per-recipient nonce effectively dead data.
- The `ModelMarketingNewsletter` declares the wrong table (`post` instead of `newsletter`).

These are documented as findings, not as recommendations. The stance is unchanged: **descriptive, not prescriptive**.

1.3 Methodology

Every claim in this paper is verified against the PHP source commit `ce75adf`. The relevant files (41 files, ~17,000 LOC):

- 6 admin marketing controllers (3,676 lines)
- 5 admin marketing models (750 lines)

- 4 storefront marketing files (278 lines)
- 7 cron files including `cron.php` (1,493 lines)
- 9 email library files (7,684 lines) + 2 extras (`utf8.php`, `vcard.php`, 1,418 lines)
- 10 admin API v1.0.0 marketing files (874 lines)
- 13 marketing-related database tables (from `necoyoad_db.sql`)

1.4 Document Roadmap

Section 2 gives a 30-second overview. Section 3 covers the 9 marketing tables. Section 4 covers newsletter composition. Section 5 covers contacts and contact lists. Section 6 covers the campaign composer and commit phases. Section 7 covers the cron sender (the most important file). Section 8 covers the tracking pixel and link redirect. Section 9 covers the PHPMailer wrapper and SMTP. Section 10 covers the vestigial bounce processor. Section 11 covers the birthday/promoter/seller tasks. Section 12 covers the spam rules. Section 13 covers the admin API marketing endpoints. Section 14 is the heart: a 6-stage campaign lifecycle trace. Section 15 catalogues the 26 cross-cutting findings. Appendix A lists the full marketing file inventory.

2 Thirty-Second Overview

The admin composes a newsletter (HTML body with `{%placeholder%}` personalisation tokens). The admin creates a campaign that targets one or more contact lists, picks a mailserver, sets a schedule, and toggles tracking flags. The campaign composer parses the newsletter HTML with `DOMDocument`, optionally embeds images as base64, appends a tracking-pixel `<img>`, rewrites every `<a href>` to a trackable `marketing/campaign/link` URL, scores the body against spam rules, and caches the rewritten HTML.

The admin commits the campaign. The commit phase inserts a `campaign` row, expands the contact lists into `campaign_contact` rows, inserts `campaign_link` rows (one per recipient per link), inserts a `task` row with `params['job'] = 'send_campaign'`, and inserts `task_queue` rows (one per recipient). The per-recipient rewritten HTML is cached.

A Unix `cron` entry runs `system/cron/cron.php` every few minutes. The cron picks up due `task` rows, hydrates their `task_queue` rows, and dispatches to `CronSend::sendCampaign()`. The sender acquires a lock via `task_exec`, loads the campaign + newsletter + mailserver config, loops the queue (max 50 per pass), and for each recipient: reads the per-recipient cached HTML (or rebuilds it if missing), configures a `Mailer` (PHPMailer 5.0) with the mailserver's SMTP credentials, sends the email, and marks the queue entry done. If the queue is not empty after 50 sends, the task is deferred 15 minutes.

When the recipient opens the email, their client auto-loads the tracking-pixel `<img>`, which hits `marketing/campaign/trace`. The trace endpoint inserts a `campaign_stat` row (with full HTTP context: serialised `$_SERVER`, `$_SESSION`, `$_REQUEST`, browser, OS, IP, referrer) and streams a 1x1 JPEG from `necoyoad.com`.

When the recipient clicks a link, they hit `marketing/campaign/link`, which inserts a `campaign_link_stat` row (with the same HTTP context plus the link nonce), looks up the original URL from `campaign_link` by the nonce, and 302-redirects.

3 The Marketing Database Schema

The marketing subsystem uses 9 tables (plus the shared `task`, `task_queue`, `task_exec`, `property`, and `setting` tables).

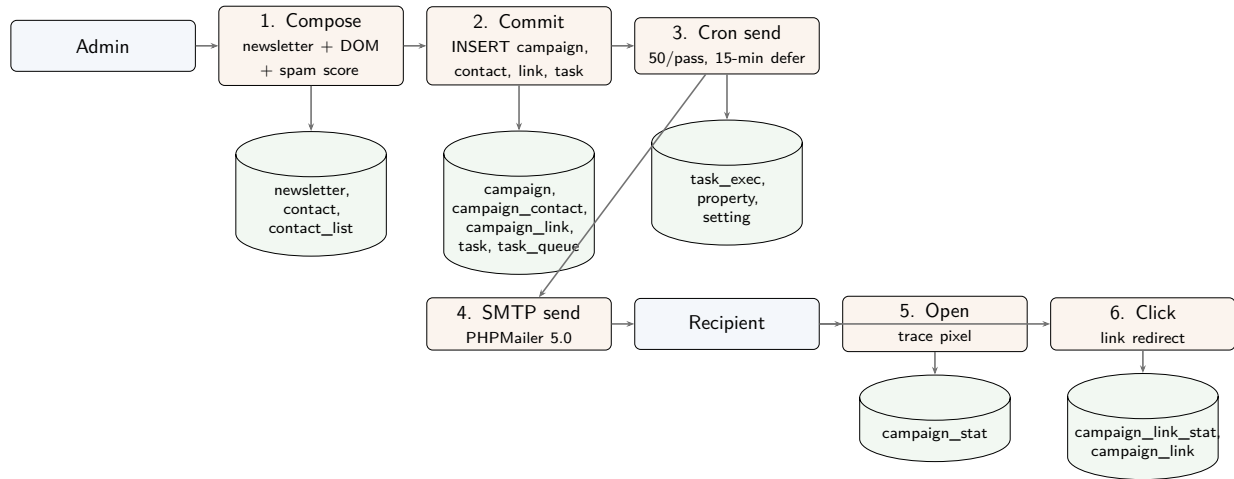


Figure 1: The 6-stage campaign pipeline at a glance. Stage 1–2 are admin-side (compose + commit). Stage 3–4 are cron-side (send + SMTP). Stage 5–6 are recipient-side (open pixel + click redirect). The full lifecycle trace is in Section 14.

3.1 The 9 Marketing Tables

Table 1: The 9 marketing tables.

Table	Primary key	Purpose
newsletter	newsletter_id	Reusable email body (name, textbody, htmlbody, status, dates).
campaign	campaign_id	A sending of a newsletter (newsletter_id, name, subject, from_name, from_email, replyto_email, trace_email, trace_click, embed_image, repeat, date_start, date_end).
campaign_contact	campaign_contact_id	One recipient of one campaign (campaign_id, contact_id, task_queue_id, name, email, status).
campaign_link	campaign_link_id	Trackable link (campaign_id, url, redirect, link, date_added).
campaign_link_stat	..._stat_id	Link-click event (campaign_id, contact_id, customer_id, link, full HTTP context).
campaign_stat	campaign_stat_id	Email-open event (campaign_id, contact_id, customer_id, full HTTP context).
contact	contact_id	CRM contact (customer_id, name, email, telephone, provider, is_active, is_deleted).
contact_list	contact_list_id	Named list of contacts (name, description, status, dates).
contact_to_list	..._list_id	Junction: contact ↔ list (unique on (contact_list_id, contact_id)).

3.2 Notable Schema Details

3.2.1 The campaign_contact.task_queue_id Column

The schema declares a `task_queue_id` column on `campaign_contact`, suggesting a foreign-key relationship between a recipient and their queued send task. The source reveals this column is *never written*. The relationship between a campaign recipient and their `task_queue` row is implicit (matching `campaign_id` + `contact_id` in the queue row’s serialised `params`).

3.2.2 The campaign_link Dual-Column URL Storage

`campaign_link` stores two URL columns:

- `url` — the rewritten tracking URL (`.../link&...&link_index=<md5>`).
- `redirect` — the original href from the newsletter HTML (e.g. `https://shop.example/spring`).
- `link` — the md5 nonce used as the lookup key.

The `link redirect` endpoint looks up the original URL via `SELECT DISTINCT redirect FROM campaign_link WHERE link = $link_index`. Notably, this lookup is *not* scoped by `campaign_id` — any campaign's link nonce can be resolved by any other campaign's redirect handler.

3.2.3 The Telemetry Tables

`campaign_stat` and `campaign_link_stat` both record a remarkably rich HTTP context per event: `store_url`, `server` (serialised `$_SERVER`), `session` (serialised `$_SESSION`), `request` (serialised `$_REQUEST`), `ref` (referrer), `browser`, `browser_version`, `os`, `ip`, `date_added`. This is the same forensic HTTP-context snapshot pattern v1 identified in the `stat`, `search`, and `user_activity` tables.

3.2.4 The Mailserver Storage Overload

There is no dedicated `mailserver` table. Mailserver credentials are stored in the `setting` table under group `mail_server`, with the key being a unique ID (e.g. `abc123def4`) and the value being a serialised PHP array of `server`, `port`, `security`, `username`, `password`. The password is base64-encoded (not encrypted).

Per-campaign mailserver selection is stored in the polymorphic property EAV table: `object_type='campaign'`, `group='mail_server'`, `key='mail_server_id'`, `value=serialize($mail_server_id)`.

4 Newsletter Composition

A newsletter is a reusable email body. The admin creates one via `marketing/newsletter/insert`.

4.1 The Newsletter Controller

`app/admin/controller/marketing/newsletter.php` (794 lines) handles newsletter CRUD. The `insert()` method is the most interesting:

```
1 public function insert() {
2     if (!$this->hasPermission('modify', 'marketing/newsletter')) {
3         return $this->forward('error/permission');
4     }
5     if ($this->request->isPost() && $this->validateForm()) {
6         // 1. Parse the htmlbody with DOMDocument
7         $dom = new DOMDocument;
8         $dom->loadHTML($this->request->post['htmlbody']);
9
10        // 2. Extract base64-encoded images, write them to DIR_IMAGE/data/
11        $images = $dom->getElementsByTagName('img');
12        foreach ($images as $image) {
13            $src = $image->getAttribute('src');
14            if (strpos($src, 'data:') === 0 && strpos($src, 'base64,') !==
15                false) {
16                list(, $data) = explode('base64,', $src);
17                $ext = substr($src, strpos($src, '/') + 1, strpos($src, ';')
18                    - strpos($src, '/') - 1);
19                $filename = slugify($this->request->post['name']) . '-' .
20                    uniqid() . '_' . time() . '.' . $ext;
21                file_put_contents(DIR_IMAGE . 'data/' . $filename,
22                    base64_decode($data));
23                $image->setAttribute('src', HTTP_IMAGE . 'data/' . $filename);
24            }
25        }
26
27        // 3. htmlentities the result and save
28        $this->request->post['htmlbody'] = htmlentities($dom->saveHTML());
29        $this->load->model('marketing/newsletter');
30        $this->modelNewsletter->add($this->request->post);
31    }
```



```

27         $this->redirect('marketing/newsletter');
28     }
29     // ... render form ...
30 }

```

Listing 1: ControllerMarketingNewsletter::insert() — newsletter creation with base64 image extraction (abridged).

The WYSIWYG editor (used in the form) lets the admin paste images directly, which the browser embeds as `data:image/...;base64,...`. The controller extracts these, writes them to `DIR_IMAGE/data/`, and replaces the `src` with an HTTP URL. The result is then `htmlentities`’d and stored.

4.2 The Personalisation Tokens

The newsletter HTML body can contain `{%placeholder%}` tokens that are substituted at send time. The cron sender (§7) substitutes:

Table 2: The 7 personalisation tokens substituted per recipient.

Token	Source	Example
<code>{%contact_id%}</code>	<code>params['contact_id']</code>	101
<code>{%campaign_id%}</code>	<code>params['campaign_id']</code>	7
<code>{%fullname%}</code>	<code>params['name']</code>	Alice Smith
<code>{%rif%}</code>	<code>params['rif']</code>	J-12345678-9
<code>{%company%}</code>	<code>params['company']</code>	Acme C.A.
<code>{%email%}</code>	<code>params['email']</code>	alice@x.com
<code>{%telephone%}</code>	<code>params['telephone']</code>	+58 212 555 1212

Plus the `prepareTemplate()` method substitutes 13 store-level tokens:

Table 3: The 13 store-level template tokens.

Token	Source	Substitution
<code>{%store_logo%}</code>	<code>config_logo</code>	<code></code>
<code>{%store_url%}</code>	<code>HTTP_HOME</code>	URL
<code>{%url_login%}</code>	<code>Url::createUrl('account/login')</code>	URL
<code>{%store_owner%}</code>	<code>config_owner</code>	string
<code>{%store_name%}</code>	<code>config_name</code>	string
<code>{%store_rif%}</code>	<code>config_rif</code>	string
<code>{%store_email%}</code>	<code>config_email</code>	string
<code>{%store_telephone%}</code>	<code>config_telephone</code>	string
<code>{%store_address%}</code>	<code>config_address</code>	string
<code>{%date_added%}</code>	<code>date('Y-m-d')</code>	date
<code>{%ip%}</code>	<code>\$_SERVER['REMOTE_ADDR']</code>	IP
<code>{%qr_code_store%}</code>	BarcodeQR for <code>HTTP_HOME</code>	<code><img src="cache/<slug>.png"></code>
<code>{%barcode_39_order_id%}</code>	Barcode39 for <code>C_CODE</code>	<code><img src="cache/<slug>_barcode_39.gif"></code>

A “Powered By Necotienda <year>” footer is appended to every email body.

4.3 The Model Schema Mismatch

Schema mismatch in ModelMarketingNewsletter

`app/admin/model/marketing/newsletter.php` (80 lines) declares `$table = 'post'` and `$fields` for the CMS `post` table, not the `newsletter` table. The Model base class’s generic CRUD uses these declarations. In practice, the controller’s `insert()` calls `$this->modelNewsletter->add($this->request->post)`, which (via the Model base) would attempt an `INSERT INTO post SET ...` with the newsletter fields — which would either fail or insert into the wrong table.

The storefront `app/shop/model/marketing/newsletter.php` (21 lines) has the same mismatch. This is a live bug in the source. v4 documents it; it does not propose a fix.

5 Contacts and Contact Lists

A contact is a CRM record (optionally linked to a customer via `customer_id`). A contact list is a named collection. Contacts and lists are many-to-many via `contact_to_list`.

5.1 The Contact Controller

`app/admin/controller/marketing/contact.php` (959 lines) handles contact CRUD plus a 3-step AJAX import wizard and a vCard export.

5.1.1 The Import Wizard

The import is a 3-step AJAX flow:

1. Step 1: admin uploads a CSV. The controller parses it, caches the parsed rows in `contact.import.<session>`, and renders a preview table.
2. Step 2: admin maps CSV columns to contact fields. The mapping is cached.
3. Step 3: admin confirms. The controller reads the cached rows + mapping, inserts new contacts (deduping by email), and adds them to the selected contact list.

The cache is keyed by session ID, so multiple admins can run imports concurrently without colliding.

5.1.2 The vCard Export

The vCard export generates a `.vcf` file for each contact. Notably, the cleanup step before generation:

```
1 // Delete every .vcf in DOCUMENT_ROOT as cleanup
2 foreach (glob($_SERVER['DOCUMENT_ROOT'] . '/*.vcf') as $file) {
3     unlink($file);
4 }
5 // Then generate the new .vcf files
6 foreach ($contacts as $contact) {
7     $vcard = new VCard();
8     $vcard->addName($contact['lastname'], $contact['firstname']);
9     $vcard->addEmail($contact['email']);
10    // ...
11    file_put_contents($_SERVER['DOCUMENT_ROOT'] . '/' . $contact['slug'] .
12                      '.vcf', $vcard);
13 }
```

Listing 2: The vCard export cleanup (abridged).

The cleanup deletes *every* `.vcf` in the web root before generating new ones. This is an unusual cleanup strategy.

5.2 The Contact List Controller

`app/admin/controller/marketing/list.php` (579 lines) handles contact list CRUD. A list has a name, description, and status. Contacts are added to lists via the `contact_to_list` junction (unique on (`contact_list_id`, `contact_id`) to prevent duplicates).

5.3 The ModelMarketingList Type Bug

Type bug in ModelMarketingList

`app/admin/model/marketing/list.php` declares `$fields['name']` with type `int`. The `contact_list.name` column is `VARCHAR(250)`. The Model base's `buildSQLQuery` uses the declared type to cast filter values — so filtering by name would cast the search string to `int`, breaking the query.

6 The Campaign Composer and Commit

A campaign is a sending of a newsletter to a set of contacts. The admin creates one in two phases: *compose* (preview) and *commit* (enqueue).

6.1 Phase 1 — Compose (marketing/campaign/insert)

`app/admin/controller/marketing/campaign.php::insert()` (the compose entry point) does:

1. Loads the newsletter by `newsletter_id`.
2. Parses the newsletter HTML with `DOMDocument`.
3. If `embed_image` is on, embeds each `<img>` as base64.
4. If `trace_email` is on, appends a tracking-pixel `<img>` with `src=Url::createUrl("marketing/campaign/trace", ['contact_id'=>'{%contact_id%}', 'campaign_id'=>'{%campaign_id%}'])`.
5. If `trace_click` is on, iterates `<a>` tags:
 - Generates `$link_index = md5(time() . mt_rand(1000000,9999999) . $href)` (a 32-hex-char nonce).
 - Rewrites `href` to `Url::createUrl("marketing/campaign/link", ['contact_id'=>'{%contact_id%}', 'campaign_id'=>'{%campaign_id%}', 'link_index'=>$link_index])`.
 - Collects `$links[] = ['url', 'redirect', 'link_index']` per link.
6. Loads the contacts from the selected contact lists: `SELECT *, t.date_Added AS created, t.email AS mail FROM contact t LEFT JOIN customer c ON (c.customer_id = t.customer_id) LEFT JOIN contact_to_list c2l ON (t.contact_id = c2l.contact_id) WHERE c2l.contact_list_id IN (5)`.
7. Deduplicates by email.
8. Computes `date_start_exec` and `date_end_exec` from the POST `year/month/day/hour/minute/meridium` fields.
9. `htmlentities($dom->saveHTML()) → $htmlbody`.
10. **Spam scoring:** `require_once(... 'spam_rules.php')`, iterates `$spam_rules`, computes `$score + $broken_rules`.
11. Caches `campaign.html.temp` and `campaign.data.temp` (serialised `['to'=>$to, 'post'=>$post, 'links'=>$links]`).
12. Renders the `beforeSend` preview page with stats: `total_embed_images`, `total_trace_links`, `total_images`, `total_links`, `date_start_exec`, `spam_score`, `broken_rules`, `email_size`, `contacts count`.

6.2 Phase 2 — Commit (marketing/campaign/send)

`ControllerMarketingCampaign::send()` (the commit entry point, clicked from the `beforeSend` preview):

```

1 public function send() {
2     // 1. Read the cached composed HTML and data
3     $htmlbody = html_entity_decode($this->cache->get("campaign.html.temp"));
4     $htmlbody = str_replace('%7B', '{', $htmlbody); // decode URL-encoded
        placeholders
5     $htmlbody = str_replace('%7D', '}', $htmlbody);
6     $data = unserialize($this->cache->get("campaign.data.temp"));
7     $to = $data['to']; // 3 contacts
8     $campaign = $data['post'];
9     $links = $data['links']; // 1 link
10
11     // 2. INSERT campaign row
12     $campaign['contacts'] = $to;
13     $campaign_id = $this->modelCampaign->add($campaign); // INSERT INTO
        campaign ...
14     // (the model's on('save') hook inserts campaign_contact rows)
15
16     // 3. Store the mailserver_id in the EAV property table
17     $this->modelCampaign->setProperty($campaign_id, 'mail_server',
        'mail_server_id', $campaign['mail_server_id']);

```

```

18
19 // 4. Build the Task
20 $task = new Task($this->registry);
21 $task->object_id = $campaign_id;
22 $task->object_type = 'campaign';
23 $task->task = $campaign['name'];
24 $task->type = 'send';
25 $task->time_exec = $campaign['date_start_exec'];
26 $task->params = ['job' => 'send_campaign', 'campaign_id' => $campaign_id];
27 $task->run_once = true;
28 $task->status = 1;
29
30 // 5. Per-recipient: substitute body placeholders, cache, and enqueue
31 foreach ($to as $sort_order => $contact) {
32     // Per-link: substitute {contact_id} and {campaign_id}, INSERT
33     // campaign_link
34     foreach ($links as $link) {
35         $link['url'] = str_replace('{%contact_id%}',
36             $contact['contact_id'], $link['url']);
37         $link['url'] = str_replace('{%campaign_id%}', $campaign_id,
38             $link['url']);
39         $this->modelCampaign->addLink($link, $campaign_id);
40     }
41     $params = ['contact_id'=>$contact['contact_id'],
42         'name'=>$contact['name'],
43         'email'=>$contact['email'], 'campaign_id'=>$campaign_id];
44     $queue = ['params'=>$params, 'status'=>1,
45         'time_exec'=>$campaign['date_start_exec']];
46     // Substitute the body-level placeholders
47     $htmlbody = str_replace('{%contact_id%}', $contact['contact_id'],
48         $htmlbody);
49     $htmlbody = str_replace('{%campaign_id%}', $campaign_id, $htmlbody);
50     $ck = "campaign.html.{%campaign_id%}.{%contact_id%}";
51     $this->cache->set($ck, $htmlbody);
52     $task->addQueue($queue);
53 }
54
55 // 6. INSERT task + task_queue rows
56 $task->createSendTask();
57 $this->redirect('marketing/campaign');
58 }

```

Listing 3: ControllerMarketingCampaign::send() — the commit phase (abridged).

6.3 The Double Link Rewriting

Per-recipient link rewriting happens TWICE per campaign

The compose phase (Phase 1) rewrites every `<a href>` into a trackable URL and collects the `$links` array. The commit phase (Phase 2) inserts `campaign_link` rows — one per recipient per link, all with the *same* link nonce (because the nonce was generated once in compose).

Then the cron sender (Phase 3, §7) rewrites the links *again* — it generates a *fresh* nonce per recipient per link and inserts new `campaign_link` rows.

The result: each tracked link ends up with *two* `campaign_link` rows per recipient — one from the commit phase, one from the cron. The commit-phase rows are effectively dead data because the cron's fresh nonces are what end up in the sent email.

If the per-recipient cache `campaign.html.<cid>.<contact_id>` hits, the cron skips the rewriting (and the second `campaign_link` INSERT). So the double-rewriting only manifests on cache miss.

7 The Cron Sender

`system/cron/api/send.php` (511 lines) is the most important file in the marketing subsystem. The `CronSend` class picks up due task rows whose `params['job']` is `send_campaign`, `send_birthday`, or `send_recommended_products`, and dispatches to the matching private method.

7.1 The sendCampaign Algorithm

```

1 public function sendCampaign($task) {
2     // 1. Lock check
3     if ($this->isLocked('send_campaign', $task->task_id)) {
4         $task->addMinute(15); // defer 15 min
5         return;
6     }
7
8     // 2. Acquire lock
9     $task->start(); // DELETE + INSERT INTO task_exec
10
11    // 3. Load campaign + newsletter
12    $query = $this->db->query("SELECT * FROM campaign c
13        LEFT JOIN newsletter n ON (n.newsletter_id = c.newsletter_id)
14        WHERE campaign_id = '" . (int)$task->params['campaign_id'] . "'");
15    $campaign_info = $query->row;
16
17    // 4. Load mailserver config from EAV property + setting tables
18    $ms_query = $this->db->query("SELECT * FROM property
19        WHERE object_type='campaign' AND group='mail_server'
20        AND key='mail_server_id' AND object_id = '" .
21        (int)$task->params['campaign_id'] . "'");
22    $mail_server_id = unserialize($ms_query->row['value']);
23    $srv_query = $this->db->query("SELECT * FROM setting
24        WHERE group='mail_server' AND key='" .
25        $this->db->escape($mail_server_id) . "'");
26    $mail_server = unserialize($srv_query->row['value']);
27    // $mail_server = ['server'=>'smtp.gmail.com', 'port'=>'587',
28    //                 'security'=>'tls',
29    //                 'username'=>'marketing@shop.example',
30    //                 'password'=>'app-password']
31
32    // 5. Load newsletter HTML body
33    $htmlbody = html_entity_decode($campaign_info['htmlbody']);
34
35    // 6. Per-queue loop (max 50 per pass)
36    $count = 0;
37    foreach ($task->getTaskQueue() as $key => $queue) {
38        if ($count >= 50) break;
39        $params = unserialize($queue['params']);
40
41        // 6a. Cache lookup (per-recipient rewritten HTML)
42        $cacheKey =
43            "campaign.html.{$params['campaign_id']}.{$params['contact_id']}";
44        $cached = $this->cache->get($cacheKey);
45        if ($cached) {
46            $htmlbody = html_entity_decode($cached);
47        } else {
48            // 6b. Cache miss: rebuild
49            $htmlbody = str_replace("%7B", "{", $htmlbody);
50            $htmlbody = str_replace("%7D", "}", $htmlbody);
51            $htmlbody = str_replace("{%contact_id%", $params['contact_id'],
52                $htmlbody);
53            $htmlbody = str_replace("{%campaign_id%",
54                $params['campaign_id'], $htmlbody);

```

```

48     $htmlbody = str_replace("{%fullname%}", $params['name'],
    $htmlbody);
49     $htmlbody = str_replace("{%rif%}", $params['rif'], $htmlbody);
50     $htmlbody = str_replace("{%company%}", $params['company'],
    $htmlbody);
51     $htmlbody = str_replace("{%email%}", $params['email'], $htmlbody);
52     $htmlbody = str_replace("{%telephone%}", $params['telephone'],
    $htmlbody);
53     $htmlbody = $this->prepareTemplate($htmlbody, $params);
54
55     // 6c. DOM parse, optional image embedding, tracking pixel, link
    rewriting
56     $dom = new DOMDocument;
57     $dom->loadHTML($htmlbody);
58     if ($params['embed_image']) { /* base64-embed images */ }
59     // Append tracking pixel
60     $trace_url = Url::createUrl("marketing/campaign/trace", [...],
    'NONSSL', HTTP_HOME);
61     $trackEmail = $dom->createElement('img');
62     $trackEmail->setAttribute('src', $trace_url);
63     $dom->appendChild($trackEmail);
64     // Rewrite every <a href> with a fresh nonce, INSERT campaign_link
65     foreach ($dom->getElementsByTagName('a') as $link) {
66         $href = $link->getAttribute('href');
67         if (empty($href) || $href == "#" || strpos($href, "mailto:")
            !== false) continue;
68         $link_index = md5(time() . mt_rand(1000000, 9999999) . $href);
69         $_link = Url::createUrl("marketing/campaign/link", [...],
            'NONSSL', HTTP_HOME);
70         $this->db->query("INSERT INTO campaign_link SET
            campaign_id=..., url=..., redirect=..., link=...,
            date_added=NOW()");
71         $link->setAttribute('href', $_link);
72     }
73     $htmlbody = $dom->saveHTML();
74 }
75
76 // 7. Send via PHPMailer 5.0
77 $mailer = new Mailer();
78 $mailer->IsSMTP();
79 $mailer->Host = $mail_server['server'];
80 $mailer->Port = $mail_server['port'];
81 $mailer->SMTPSecure = $mail_server['security'];
82 $mailer->Username = $mail_server['username'];
83 $mailer->Password = base64_decode($mail_server['password']);
84 $mailer->SMTPAuth = true;
85 $mailer->AddAddress($params['email'], $params['name']);
86 $mailer->IsHTML(true);
87 $mailer->SetFrom($campaign_info['from_email'],
    $campaign_info['from_name']);
88 $mailer->AddReplyTo($campaign_info['replyto_email'],
    $campaign_info['from_name']);
89 $mailer->Subject = $campaign_info['subject'];
90 $mailer->Body = $htmlbody;
91 $mailer->Send();
92 $mailer->ClearAllRecipients();
93
94 // 8. Mark queue entry done
95 $task->setQueueDone($key);
96 $count++;
97 }
98
99 // 9. End-of-loop: defer or complete
100 if (count($task->getTaskDos($task->task_id)) > 0) {

```

```

101     $task->addMinute(15); // more pending, defer 15 min
102 } else {
103     $task->setTaskDone(); // all done
104 }
105 $task->update();
106 }

```

Listing 4: CronSend::sendCampaign() — the campaign sender (abridged).

7.2 The Locking Mechanism

The lock is implemented via the `task_exec` table (MyISAM has no row-level locks, so the application uses delete-then-insert as a best-effort lock):

```

1 private function isLocked($job, $current_task_id) {
2     $query = $this->db->query("SELECT * FROM task_exec WHERE type = '" .
3         $this->db->escape($job) . "'");
4     if (count($query->rows)) {
5         $queue = $this->db->query("SELECT COUNT(*) AS total FROM task_queue
6             WHERE status = '1' AND task_id = '" . (int)$query->row['task_id']
7             . "'");
8         if ($queue->row['total'] > 0) {
9             if ($current_task_id == $query->row['task_id']) {
10                return false; // I already hold the lock
11            } else {
12                return true; // someone else holds it
13            }
14        } else {
15            // Stale lock, clear it
16            $this->db->query("DELETE FROM task_exec WHERE type = '" .
17                $this->db->escape($job) . "'");
18            return false;
19        }
20    }
21    return false;
22 }

```

Listing 5: CronSend::isLocked() — the lock check.

7.3 The 50-Email Throttle

The sender processes at most 50 queue entries per pass. If the queue is not empty after 50 sends, the task is deferred 15 minutes (`$task->addMinute(15)`), and the next cron tick picks it up. So a 1,000-recipient campaign takes $\lceil 1000/50 \rceil = 20$ passes, or roughly 5 hours at a 15-minute cron interval. This throttle is hard-coded; there is no per-mailserver rate-limit configuration.

7.4 The Per-Pass Cache

The per-recipient rewritten HTML is cached at `campaign.html.<campaign_id>.<contact_id>`. The commit phase (Phase 2) populates this cache. The cron sender reads it. On a cache hit, the cron skips the DOM parse, the personalisation substitution, the image embedding, the tracking-pixel append, and the link rewriting — it just sends the cached HTML. On a cache miss (e.g. if the cache was cleared), the cron rebuilds everything, which is where the double link rewriting manifests.

8 The Tracking Endpoints

The storefront has two endpoints that record recipient interactions: **trace** (email open) and **link** (link click).

8.1 The Tracking Pixel (marketing/campaign/trace)

```

1 public function trace() {
2     header('Cache-Control: no-cache');
3     header('Content-type: image/jpeg');
4     if ($this->request->hasQuery('campaign_id')) {
5         $this->load->model('marketing/campaign');
6         $this->modelCampaign->trackEmail(
7             $this->request->getQuery('campaign_id'),
8             $this->request->getQuery('contact_id')
9         );
10        if ($this->request->hasQuery('sendCampaign')) {
11            $this->link2Campaign($this->request->getQuery('sendCampaign'));
12        }
13    }
14    readfile('https://necoyoad.com/.../firma_necoyoad.jpg');
15 }

```

Listing 6: ControllerMarketingCampaign::trace() — the open-tracking pixel.

The endpoint:

1. Sets `Cache-Control: no-cache` and `Content-type: image/jpeg` (so the email client treats the response as an image).
2. Calls `trackEmail()`, which INSERTs a `campaign_stat` row with the `campaign_id`, `contact_id`, `customer_id` (if logged in), and the full HTTP context (serialised `$_SERVER`, `$_SESSION`, `$_REQUEST`, referrer, browser, OS, IP).
3. If `sendCampaign` query param is present, calls `link2Campaign()` (a hidden follow-up-campaign trigger, §11).
4. Streams a 1x1 JPEG from `necoyoad.com` — `readfile(...)` the pixel image.

The tracking pixel is hard-coded to a Necoyoad-CDN image

The tracking pixel image is fetched from a hard-coded URL on the Necoyoad marketing site (`necoyoad.com/.../firma_necoyoad.jpg`), not the local store. This means every tracking-pixel request makes an outbound HTTP call to `necoyoad.com`. If the Necoyoad marketing site is down, the tracking still records the open (the `campaign_stat` INSERT happens before the `readfile`), but the recipient's email client shows a broken image icon.

8.2 The Link Redirect (marketing/campaign/link)

```

1 public function link() {
2     if ($this->request->hasQuery('campaign_id')) {
3         $this->load->model('marketing/campaign');
4         $this->modelCampaign->trackLink(
5             $this->request->getQuery('campaign_id'),
6             $this->request->getQuery('contact_id'),
7             $this->request->getQuery('link_index')
8         );
9         if ($this->request->hasQuery('sendCampaign')) {
10            $this->link2Campaign($this->request->getQuery('sendCampaign'));
11        }
12        $link_index = $this->request->getQuery('link_index');
13        $redirectTo = $this->modelCampaign->getLink($link_index);
14        if ($redirectTo) {
15            $this->redirect($redirectTo);
16        } else {
17            $this->redirect(HTTP_HOME);
18        }
19    }
20 }

```

Listing 7: ControllerMarketingCampaign::link() — the click-tracking redirect.

The endpoint:

1. Calls `trackLink()`, which INSERTs a `campaign_link_stat` row with the `campaign_id`, `contact_id`, `customer_id`, the link nonce, and the full HTTP context.
2. If `sendCampaign` query param is present, calls `link2Campaign()`.
3. Looks up the original URL via `getLink()`: `SELECT DISTINCT redirect FROM campaign_link WHERE link = $link_index`.
4. 302-redirects to the original URL (or to `HTTP_HOME` if not found).

The `campaign_link` lookup is not scoped by `campaign_id`

`getLink($link)` does `SELECT DISTINCT redirect FROM campaign_link WHERE link = $link`. The lookup is by nonce only, not by `campaign_id`. This means a nonce from one campaign can resolve to a redirect from a different campaign (if the nonces collide, which is unlikely with md5 but not impossible). It also means the lookup returns the first matching row, which (per the double-rewriting finding) could be either the commit-phase row or the cron-phase row — but since both have the same `redirect` value, the result is correct.

8.3 The `trackEmail` SQL Bug

`trackEmail` and `trackLink` use `INSERT` not `INSERT INTO`

The `ModelMarketingCampaign::trackEmail()` and `trackLink()` methods use `INSERT <table> SET ...` instead of the standard `INSERT INTO <table> SET ....` MySQL tolerates `INSERT` as a synonym for `INSERT INTO`, so this works in practice, but it is non-standard SQL and would fail on strict SQL modes or other databases.

9 The Mailer (PHPMailer 5.0)

The campaign sender uses `system/library/email/mailer.php` (2,289 lines), which is PHPMailer 5.0 (released 2009) with the class renamed from `PHPMailer` to `Mailer`.

9.1 Two Parallel Mailers

Two parallel hand-rolled mailers coexist

The codebase has two parallel mailer stacks:

- `system/library/email/mailer.php` (2,289 lines) — PHPMailer 5.0, class `Mailer`. Used by the cron sender for campaigns.
- `system/library/mail.php` (397 lines) — a separate, simpler `Mail` class. Used by the order-confirmation email flow (`checkout/success`).
- `system/library/ntsmailer.php` (164 lines) — `ntsMailer`, a thin wrapper that picks between the two based on config.

Different parts of the codebase use different mailers. The campaign subsystem always uses `Mailer` (PHPMailer 5.0).

9.2 The SMTP Client

`system/library/email/smtp.php` (814 lines) is a from-scratch SMTP client (not a wrapper around PHP's `mail()` or a third-party library). It supports:

- `AUTH LOGIN` (base64-encoded username and password).
- `STARTTLS` (for `SMTPSecure = 'tls'`).
- `SMTPS` (for `SMTPSecure = 'ssl'`).
- Multiple recipients, CC, BCC, attachments, embedded images.

It does *not* support AUTH CRAM-MD5, AUTH PLAIN, or AUTH XOAUTH2. So campaigns can only be sent via SMTP servers that accept AUTH LOGIN with a username+password (e.g. Gmail with an app password, but not Gmail with OAuth).

9.3 The POP3 Client (for Bounce)

`system/library/email/pop3.php` (406 lines) is a from-scratch POP3 client. It is used by the (vestigial) bounce processor (§10).

9.4 The Password Storage

Mailserver passwords are stored in the `setting` table, base64-encoded (not encrypted). The cron sender decodes them with `base64_decode($mail_server['password'])` before passing to `Mailer`. This is encoding, not encryption — anyone with database read access has all mailserver credentials.

10 The Vestigial Bounce Processor

Bounce processing is entirely vestigial

`system/cron/api/bounce.php` (167 lines) is **not loaded** by `cron.php` — the `require_once` for `bounce.php` is commented out at line 17. The file's contents reference `SENDSTUDIO_TABLEPREFIX`, `SENDSTUDIO_API_DIRECTORY`, `Jobs_Bounce_API`, `Jobs_API`, `CheckCronSchedule`, `GetLang`, `common.php` — none of which exist in the Necoyoad codebase. It is a verbatim copy of Interspire SendStudio's `admin/cron/bounce.php` and is effectively dead code. It would fatal-error on `require_once dirname(__FILE__) . '/common.php'` (which doesn't exist). The function `Cron_BounceProcess` (lines 44–165) is a docstring-only description of the bounce-processing flow it would perform if the Interspire API were present.

The admin-side `app/admin/model/marketing/bounce.php` (194 lines) targets a non-existent `email_lists` schema (another Interspire relic). It defines methods like `getBounces()`, `deleteBounce()`, `processBounce()` — but the tables they query (`email_lists`, `email_list_subscriptions`, `email_queue`) do not exist in the Necoyoad schema.

The platform has *no working bounce detection*. Bounced emails are not automatically unsubscribed; the contact stays active and will be emailed again on the next campaign. The only way to stop sending to a bounced address is for the admin to manually mark the contact as `is_deleted` or `is_active = 0`.

11 The Auxiliary Tasks: Birthday, Promoter, Seller

The cron dispatches three other marketing-related tasks alongside `send_campaign`.

11.1 CronBirthday (system/cron/api/birthday.php, 97 lines)

Sends birthday greeting emails. The cron queries `SELECT * FROM customer WHERE birthday = TODAY() AND congrats = 1`, loads the newsletter specified by the `marketing_mailserver_happy_birthday` config key, and for each birthday customer, substitutes `{%fullname%}`, `{%rif%}`, `{%company%}`, `{%email%}`, `{%telephone%}` and sends via `Mailer`.

Birthday congrats-reset has an AND/OR semantic bug

After sending, the cron sets `customer.congrats = 0` to prevent re-sending next year. The query is: `UPDATE customer SET congrats = 0 WHERE customer_id = X AND congrats = 1 OR customer_id = X`. Due to operator precedence (AND binds tighter than OR), this parses as `(customer_id = X AND congrats = 1) OR (customer_id = X)` — which simplifies to `customer_id = X`. So the update sets `congrats = 0` for the customer regardless of the current value. This happens to be the desired behaviour (always reset), but the intent was clearly to only reset if currently 1.

11.2 CronPromoter (system/cron/api/promoter.php, 167 lines)

Sends “promoter” emails — product recommendations based on the customer’s referral activity. Queries the `stat` table for today’s product views, picks the top N, and sends a “recommended products” email to each promoter (customer with `referenced_by > 0`).

11.3 CronSeller (system/cron/api/seller.php, 92 lines)

CronSeller is a stub

`CronSeller` is a stub. The `run()` method is empty. It would fatal-error if instantiated (it references undefined properties) or never be instantiated (the cron constructor does not instantiate it).

11.4 CronUpdate (system/cron/api/update.php, 214 lines)

CronUpdate is a stub

`CronUpdate` is also a stub. It is not instantiated by the cron constructor. Its `run()` method contains commented-out logic for updating campaign statuses (marking completed campaigns, advancing scheduled ones).

11.5 The Hidden link2Campaign Mechanism

The storefront tracking endpoints (`trace` and `link`) have a hidden `sendCampaign` query parameter. If present, the endpoint calls `link2Campaign($campaign_id)`, which:

1. Loads the contact and the source campaign.
2. Copies the campaign with name suffix “ - Campaign Triggered”, sets `date_start` to `now + 15 minutes` and `date_end` to `now + 2 days`, sets `repeat = ""`.
3. Targets only the current contact.
4. Mirrors the `ControllerMarketingCampaign::send` logic: `modelCampaign->add + setProperty + new Task + addQueue + createSendTask`.

This is a follow-up-campaign trigger: when a recipient opens (or clicks) a campaign, a follow-up campaign can be automatically scheduled for them 15 minutes later. No admin UI exposes this mechanism — it would have to be triggered by manually crafting a URL with `?sendCampaign=<id>`. It could be used for forward-to-a-friend (the recipient forwards the email, the friend’s open triggers a follow-up campaign), but no such feature is implemented.

12 The Spam Rules

`system/library/email/spam_rules.php` (1,177 lines) is a content-scoring system applied to the campaign HTML body at compose time. It returns a numeric score and a list of broken rules.

12.1 The Three-Layer Spam Pipeline

Three layers of spam-rule application

The spam rules are applied in three layers:

1. `spam_rules.php` defines a global `$spam_rules` array of rule definitions (each with a regex/pattern, a score, and a description).
2. The campaign composer (`insert()`) iterates `$spam_rules`, applies each rule to the HTML body, and accumulates `$score + $broken_rules`.
3. The `beforeSend` preview displays the score and the broken rules, so the admin can see if the email is likely to be flagged as spam.

The rules themselves are a mix of content checks (e.g. “too many exclamation marks”, “all-caps subject”, “spammy words like FREE, GUARANTEE, ACT NOW”) and structural checks (e.g. “image-to-text ratio too high”, “too many colours”, “javascript present”).

12.2 The Rule Format

Each rule in `$spam_rules` is an array with:

- **pattern** — a regex or string to match.
- **score** — the numeric penalty if matched.
- **description** — a human-readable explanation.
- **category** — e.g. `content`, `structure`, `subject`.

The total score is the sum of all matched rules' scores. A score above a threshold (configurable) is flagged as “likely spam”. The admin can still send the campaign, but they are warned.

13 The Admin API Marketing Endpoints

The admin API v1.0.0 has 5 marketing resource endpoints (each with a paired `_data.php` file), following the same pseudo-REST shape as the other API endpoints (v2 §7).

13.1 The 5 Endpoints

Table 4: The 5 admin API v1.0.0 marketing endpoints.

Resource	Files	Operations
campaigns	campaigns.php (154) + _data.php (42)	GET (list with filters), POST (create), PUT (update), DELETE.
contacts	contacts.php (151) + _data.php (32)	Same.
contact_lists	contact_lists.php (149) + _data.php (31)	Same.
mailservers	mailservers.php (102) + _data.php (34)	Same.
newsletters	newsletters.php (147) + _data.php (32)	Same.

13.2 The Broken PUT Branches

All 5 marketing API PUT branches are broken in different ways

The PUT (update) branches of all 5 marketing API endpoints are broken:

- `campaigns.php` PUT: uses the loaded DB row as the data source instead of the JSON body.
- `contacts.php` PUT: same.
- `contact_lists.php` PUT: same.
- `mailservers.php` PUT: references an undefined `$query` variable (would fatal).
- `newsletters.php` PUT: same as campaigns.

The GET, POST, and DELETE branches work. Only PUT is broken. The `prepareData()` call in each PUT branch receives the existing DB row instead of `$this->request->post`, so the update is a no-op (or, for mailservers, a fatal).

14 The Campaign Lifecycle Trace

This trace walks through a single campaign lifecycle: the admin creates a newsletter, creates a campaign targeting 3 contacts, the cron sends it, one recipient opens it, and that recipient clicks the link.

14.1 Stage 1 — Newsletter Create

Entry: `POST /admin/index.php?r=marketing/newsletter/insert` with `name="Spring Sale"`,
`htmlbody="<html><body><h1>Hi {%fullname%}</h1><p>Welcome {%email%}</p><a href='https://shop.example/spring'>Shop now</a></body></html>".`

1. `ControllerMarketingNewsletter::insert()`.
2. Permission check: `hasPermission('modify', 'marketing/newsletter')`.
3. Parse `htmlbody` with `DOMDocument`.
4. Iterate `<img>` tags: none have `base64 src`, skip.
5. `htmlentities($dom->saveHTML())` → `post['htmlbody']`.
6. `modelNewsletter->add($post)` → `INSERT INTO newsletter SET name='Spring Sale', textbody="", htmlbody=..., date_added=NOW(), date_modified=NOW(), status=1`. Returns `newsletter_id = 42`.
DB writes: `newsletter` (1 row).

14.2 Stage 2 — Campaign Compose

Entry: `POST /admin/index.php?r=marketing/campaign/insert` with `name="April Spring Blast", newsletter_id=42, subject="Spring Sale Now On", from_name="NecoShop", from_email="marketing@shop.example", replyto_email="reply@shop.example", contact_list=5, mail_server_id="abc123def4", trace_email=1, trace_click=1, embed_image=0, schedule=2025-04-15 10:00 AM`.

1. `ControllerMarketingCampaign::insert()`.
2. Load newsletter 42: `SELECT * FROM newsletter WHERE newsletter_id = 42`.
3. Parse HTML with `DOMDocument`.
4. `trace_email=1` → append tracking-pixel ``.
5. `trace_click=1` → rewrite `<a href>`:
 - `$link_index = md5(time() . mt_rand(1000000,9999999) . 'https://shop.example/spring')` → e.g. `a1b2c3d4e5f6...`
 - Rewrite href to `.../link?contact_id={%contact_id%}&...&link_index=<nonce>`.
 - Collect a `$links` entry with the rewritten url, the original redirect, and the `link_index` nonce.
6. Load contacts from list 5: `SELECT *, t.date_Added AS created, t.email AS mail FROM contact t LEFT JOIN customer c ON (...) LEFT JOIN contact_to_list c2l ON (...) WHERE c2l.contact_list_id IN (5)`. Returns 3 contacts: Alice (101), Bob (102), Carol (103).
7. Spam scoring: iterate `$spam_rules`, compute `$score = 0.5, $broken_rules = []` (low score, no broken rules).
8. Cache: `campaign.html.temp = htmlentities($dom->saveHTML()), campaign.data.temp = serialize(['to'=>$to, 'post'=>$post, 'links'=>$links])`.
9. Render `beforeSend` preview with stats.

DB reads: `newsletter` (1), `contact + contact_to_list + customer` (1 JOIN). **Cache writes:** `campaign.html.temp, campaign.data.temp`.

14.3 Stage 3 — Campaign Commit

Entry: `POST /admin/index.php?r=marketing/campaign/send` (clicked from the `beforeSend` preview).

1. `ControllerMarketingCampaign::send()`.
2. Read cache: `$htmlbody = html_entity_decode(cache.get("campaign.html.temp"))`. Decode `%7B/%7D` → `{/}`.
3. `$data = unserialize(cache.get("campaign.data.temp"))`. `$to = [Alice, Bob, Carol]`, `$links = [spring_link]`.
4. `campaign_id = modelCampaign->add($campaign)` → `INSERT INTO campaign SET newsletter_id=42, name='April Spring Blast', subject=..., from_name=..., trace_email=1, trace_click=1, embed_image=0, date_start='2025-04-15 10:00:00', ...`. Returns `campaign_id = 7`.
5. The model's `on('save')` hook fires `setContacts(7, $to)` → `DELETE FROM campaign_contact WHERE campaign_id=7` then `INSERT INTO campaign_contact SET campaign_id=7, contact_id=101, name='Alice', email='alice@x.com', status=1` × 3.
6. `modelCampaign->setProperty(7, 'mail_server', 'mail_server_id', 'abc123def4')` → `DELETE FROM property WHERE ...` then `INSERT INTO property SET object_type='campaign', object_id=7, group='mail_server', key='mail_server_id', value=serialize('abc123def4')`.

7. Build Task: `object_id=7, object_type='campaign', task='April Spring Blast', type='send', time_exec='2025-04-15 10:00:00', params=['job'=>'send_campaign', 'campaign_id'=>7], run_once=1, status=1.`
8. Per recipient (Alice):
 - Per link: `str_replace('{%contact_id%}', 101, $link['url']), str_replace('{%campaign_id%}', 7, $link['url'])`, then `modelCampaign->addLink($link, 7) → INSERT INTO campaign_link SET campaign_id=7, url='.../link?contact_id=101&...', redirect='https://shop.example/spring', link='<nonce>', date_added=NOW().`
 - `str_replace('{%contact_id%}', 101, $htmlbody),` `str_replace('{%campaign_id%}', 7, $htmlbody).`
 - `cache.set("campaign.html.7.101", $htmlbody).`
 - `task->addQueue(['params'=>['contact_id'=>101, 'name'=>'Alice', 'email'=>'alice@x.com', 'campaign_id'=>7], 'status'=>1, 'time_exec'=>'2025-04-15 10:00:00']).`
9. Same for Bob (102) and Carol (103).
10. `task->createSendTask() → INSERT INTO task SET store_id=0, object_id=7, object_type='campaign', task='April Spring Blast', type='send', params=serialize(['job'=>'send_campaign', 'campaign_id'=>7]), time_exec='2025-04-15 10:00:00', run_once=1, status=1. Returns task_id = 99.`
11. Per queue entry: `INSERT INTO task_queue SET task_id=99, params=serialize(['contact_id'=>101, ...]), time_exec='2025-04-15 10:00:00', status=1, sort_order=0 × 3.`

DB writes: campaign (1), campaign_contact (3), campaign_link (3 — one per recipient, same nonce), property (1), task (1), task_queue (3). Cache writes: campaign.html.7.101, .102, .103.

14.4 Stage 4 — Cron Send

Entry: `php /path/to/system/cron/cron.php` (run by system crontab at 10:00).

1. `Cron::__construct():` bootstrap, load setting for `store_id=0`, init mailer.
2. `SELECT * FROM task WHERE date_start_exec <= NOW() AND time_exec <= NOW() AND status=1 ORDER BY sort_order ASC, time_exec ASC.` Returns task 99.
3. `SELECT * FROM task_queue WHERE task_id=99 AND status=1 ORDER BY sort_order ASC, time_exec ASC.` Returns 3 rows.
4. Hydrate Task object with queue.
5. `CronSend::run([task 99]) → sendCampaign(task 99).`
6. `isLocked('send_campaign', 99) → SELECT * FROM task_exec WHERE type='send_campaign' → no rows → return false.`
7. `task->start() → DELETE FROM task_exec WHERE type='send_campaign', INSERT INTO task_exec SET task_id=99, type='send_campaign', status=1, date_added=NOW().`
8. `SELECT * FROM campaign c LEFT JOIN newsletter n ON (...) WHERE campaign_id=7 → campaign + newsletter row.`
9. `SELECT * FROM property WHERE object_type='campaign' AND group='mail_server' AND key='mail_server_id' AND object_id=7 → value='s:10:"abc123def4";'.`
10. `$mail_server_id = unserialize(...) = 'abc123def4'.`
11. `SELECT * FROM setting WHERE group='mail_server' AND key='abc123def4' → value=serialize(['server'=>'smtp.gmail.com', 'port'=>'587', 'security'=>'tls', 'username'=>'marketing@shop.example', 'password'=>base64('app-password')]).`
12. Per queue entry (Alice, `contact_id=101`):
 - `$cached = cache.get("campaign.html.7.101") → HIT. $htmlbody = html_entity_decode($cached).`
 - Build Mailer: `IsSMTP(), Host='smtp.gmail.com', Port=587, SMTPSecure='tls', Username='marketing@shop.example', Password=base64_decode('app-password'), SMTPAuth=true.`
 - `AddAddress('alice@x.com', 'Alice'), IsHTML(true), SetFrom('marketing@shop.example', 'NecoShop'), AddReplyTo('reply@shop.example', 'NecoShop'), Subject='Spring Sale Now On', Body=$htmlbody.`
 - `Send() → SMTP to smtp.gmail.com:587, STARTTLS, AUTH LOGIN, MAIL FROM, RCPT TO alice@x.com, DATA (sends the email with the tracking pixel and rewritten link). Returns true.`

- `ClearAllRecipients()`.
- `task->setQueueDone(key) → UPDATE task_queue SET status=0 WHERE task_queue_id=<TQ1>`.

13. Same for Bob (TQ2) and Carol (TQ3). `$count = 3`.

14. `count(task->getTaskDos(99)) → SELECT * FROM task_queue WHERE status=1 AND task_id=99 → 0 rows → task->setTaskDone(99) → UPDATE task SET status=0 WHERE task_id=99, DELETE FROM task_exec WHERE task_id=99`.

DB writes: `task_exec` (1 INSERT + 1 DELETE), `task_queue` (3 UPDATES to `status=0`), `task` (1 UPDATE to `status=0`). **DB reads:** `setting`, `task`, `task_queue`, `campaign` + `newsletter` (JOIN), `property`, `setting` (again, for mailserver). **Cache reads:** `campaign.html.7.101/.102/.103` (all hit). **Network:** SMTP to `smtp.gmail.com:587` (3 emails).

14.5 Stage 5 — Email Open (Tracking Pixel)

Entry: Alice's email client auto-loads ``.

1. `Storefront index.php → ControllerMarketingCampaign::trace()`.
2. `header('Cache-Control: no-cache');`; `header('Content-type: image/jpeg');`;
3. `modelCampaign->trackEmail(7, 101) → INSERT campaign_stat SET campaign_id=7, contact_id=101, customer_id=0, server=serialize($_SERVER), session=serialize($_SESSION), request=serialize($_REQUEST), store_url='...', ref='<HTTP_REFERER>', browser='Chrome', browser_version='120', os='Windows', ip='<REMOTE_ADDR>', date_added=NOW()`.
4. No `sendCampaign` param, skip `link2Campaign`.
5. `readfile('https://necoyoad.com/web/assets/images/data/firma_necoyoad.jpg')` → streams the 1x1 JPEG.

DB writes: `campaign_stat` (1 row). **Network:** HTTP GET to `necoyoad.com` (the pixel image).

14.6 Stage 6 — Email Click (Link Redirect)

Entry: Alice clicks `.../link?...contact_id=101&campaign_id=7&link_index=<nonce>`.

1. `Storefront index.php → ControllerMarketingCampaign::link()`.
2. `modelCampaign->trackLink(7, 101, 'a1b2c3d4e5f6...') → INSERT campaign_link_stat SET campaign_id=7, contact_id=101, customer_id=0, link='a1b2c3d4e5f6...', server=serialize($_SERVER), session=serialize($_SESSION), request=serialize($_REQUEST), store_url='...', ref='<HTTP_REFERER>', browser='Chrome', browser_version='120', os='Windows', ip='<REMOTE_ADDR>', date_added=NOW()`.
3. No `sendCampaign` param, skip `link2Campaign`.
4. `redirectTo = modelCampaign->getLink('a1b2c3d4e5f6...') → SELECT DISTINCT redirect FROM campaign_link WHERE link='a1b2c3d4e5f6...' → 'https://shop.example/spring'`.
5. `this->redirect('https://shop.example/spring')` → 302 redirect.

DB writes: `campaign_link_stat` (1 row). **DB reads:** `campaign_link` (1 SELECT DISTINCT). **Network:** 302 redirect to `https://shop.example/spring`.

14.7 Lifecycle Summary

15 Cross-Cutting Findings

The 26 cross-cutting findings, grouped by theme.

15.1 Architecture

- **PHPMailer 5.0 (2009)** is the SMTP workhorse, renamed `PHPMailer` → `Mailer`. AUTH LOGIN only.
- Two parallel hand-rolled mailers coexist (`Mailer` + `Mail` + `ntsMailer` wrapper).
- The `setting` table is overloaded for mailserver storage — no dedicated `mailserver` table.
- Two cron dispatchers coexist: `cron.php` (active) and the Interspire-style `bounce.php` (dead).
- The `cron initConfig` only loads `store_id = 0` settings — per-store mailserver configs are not visible to the cron.

Table 5: The 6-stage campaign lifecycle summary.

Stage	Tables written	Tables read
1. Newsletter create	newsletter	—
2. Campaign compose	— (cache only)	newsletter, contact + contact_to_list + customer
3. Campaign commit	campaign, campaign_contact, campaign_link, property, task, task_queue	— (cache only)
4. Cron send	task_exec, task_queue, task	setting, task, task_queue, campaign + newsletter, property, setting (mailserver)
5. Email open	campaign_stat	—
6. Email click	campaign_link_stat	campaign_link

15.2 Bugs and Vestigial Code

- **Bounce processing is entirely vestigial** — commented out of cron, references non-existent Interspire classes.
- **CronSeller and CronUpdate are stubs** — empty `run()` methods or never instantiated.
- **Schema mismatch in ModelMarketingNewsletter** — declares `$table = 'post'`, not `'newsletter'`.
- **ModelMarketingList::\$fields['name'] is mis-typed as int** — should be `varchar`.
- **The birthday congrats-reset has an AND/OR semantic bug** — `WHERE customer_id = X AND congrats = 1 OR customer_id = X` parses as `customer_id = X`.
- **trackEmail and trackLink use INSERT not INSERT INTO** — works in MySQL but non-standard.
- **All 5 marketing API PUT branches are broken** — use loaded DB row instead of JSON body, or reference undefined `$query`.
- **vCard export deletes every .vcf in DOCUMENT_ROOT** as cleanup — unusual strategy.

15.3 Missing Features

- **No unsubscribe mechanism** — no `?unsubscribe=1` endpoint, no `List-Unsubscribe` header.
- **No forward-to-a-friend endpoint** — the hidden `link2Campaign` mechanism could enable it but no admin UI exposes it.
- **No working bounce detection** — bounced emails are not automatically unsubscribed.

15.4 Design Choices

- **The tracking pixel is hard-coded to a Necoyoad-CDN image** — every open makes an outbound HTTP call to `necoyoad.com`.
- **Venezuelan locale hard-coding** — `America/Caracas` in the campaign form, the storefront `link2Campaign`, and the `Cron` constructor.
- **campaign_contact.task_queue_id is declared but never written** — the recipient-to-queue relationship is implicit.
- **Per-recipient link rewriting happens TWICE per campaign** — admin commit + cron send, with the commit-phase rows effectively dead.
- **The campaign_link lookup is not scoped by campaign_id** — any campaign's nonce can resolve to any other campaign's redirect.
- **CronSend throttles to 50 emails per pass** with a 15-minute defer — hard-coded, no per-mailserver rate-limit config.
- **task.time_last_exec typo-bug in send()** — references undefined `$row`, emits notice.
- **Personalisation tokens are URL-encoded by `Url::createUrl`**, requiring manual `%7B/%7D` decoding.
- **campaign_link.url is the rewritten tracking URL, redirect is the original href** — dual-column storage.
- **The campaign-sender email-build pipeline has 7 personalisation tokens + 13 template tokens** — `{%fullname%}`, `{%rif%}`, `{%company%}`, `{%email%}`, `{%telephone%}`, `{%contact_id%}`, `{%campaign_id%}` + 13 store-level tokens.

- **EmailTemplate::readPremadeTemplate** rewrites 4 forms of image references — `http://`, `https://`, relative, and `data: base64`.
- **Contact import wizard** is a 3-step AJAX flow with `serialize()`-based cache — keyed by session ID.
- **promoter.php** integrates with the `stat` table for product-view-based recommendations.
- **Three layers of spam-rule application** — `spam_rules.php` defines rules, `composer` applies them, `beforeSend` displays the score.

16 Closing Observations

16.1 The Marketing Subsystem is a Self-Hosted Mailchimp Clone

Necoyoad’s marketing subsystem is, in ambition, a self-hosted Mailchimp clone: newsletter composition, contact-list targeting, scheduled sending, open/click tracking, bounce handling. In execution, it achieves the first four and fails at the fifth (the bounce processor is vestigial). The result is a functional email-marketing pipeline that can send personalised, tracked campaigns of arbitrary size, throttled to 50 emails per 15-minute pass.

16.2 The Subsystem is Where the Task Scheduler Does Real Work

v2 introduced the `task/task_queue/task_exec` scheduler but only described it abstractly. v4 shows the scheduler in its most consequential use: a 1,000-recipient campaign is literally 1,000 queued tasks, each sending one personalised email. The 50-per-pass throttle, the 15-minute defer, the `task_exec` locking, the per-recipient cache — all of these are the scheduler’s load-bearing details, exercised at scale by every campaign send.

16.3 The Subsystem is Where v1’s Inferences are Most Incomplete

v1 inferred the marketing subsystem from the schema alone. v1 was broadly correct about the shape (newsletter → campaign → contact → tracking) but could not see:

- The double link rewriting (admin commit + cron send).
- The per-recipient cache that suppresses the second rewriting on hit.
- The vestigial bounce processor.
- The broken API PUT branches.
- The hard-coded Necoyoad-CDN tracking pixel.
- The `ModelMarketingNewsletter` schema mismatch.
- The hidden `link2Campaign` follow-up mechanism.
- The 50-email throttle and 15-minute defer.
- The `task_exec` locking algorithm.
- The PHPMailer 5.0 lineage and AUTH-LOGIN-only limitation.

These are the details that distinguish a reverse-engineered blueprint from a schema diagram. v4 fills them in.

16.4 The Subsystem is Remarkably Complete Despite Its Warts

Despite the vestigial bounce processor, the broken API PUTs, the schema mismatches, and the missing unsubscribe, the marketing subsystem is remarkably complete: it can compose newsletters with a WYSIWYG, target contact lists, schedule sends, personalise per recipient, track opens and clicks, and display stats. The admin can see, per campaign, how many emails were sent, how many were opened, and how many links were clicked. This is more than many commercial e-commerce platforms offered in 2009 (when PHPMailer 5.0 was released), and it is fully self-hosted — no third-party SaaS dependency.

16.5 Final Note

v4 confirms that the Necoyoad marketing subsystem is a characterful, hand-built email-marketing pipeline that combines OpenCart-derived conventions (the `Controller/Model` split, the `.tpl` file convention) with a proprietary campaign pipeline (the `task_queue`-driven sender, the `DOMDocument`-based link rewriting, the `campaign_stat/campaign_link_stat` telemetry, the PHPMailer 5.0 SMTP workhorse). The result is a subsystem that is flexible enough to support personalised, tracked campaigns of arbitrary size, while carrying the scars of incomplete features (bounce, unsubscribe, API PUTs) and hard-coded assumptions (Venezuelan locale, Necoyoad-CDN pixel, 50-email throttle).

A Marketing File Inventory

B Glossary

Table 6: The 41 marketing-subsystem files (~17,000 LOC).

File	Lines	Role
Admin controllers (3,676 lines)		
app/admin/controller/marketing/campaign.php	843	Campaign CRUD + compose + commit + tracking stats.
app/admin/controller/marketing/contact.php	959	Contact CRUD + 3-step CSV import wizard + vCard export.
app/admin/controller/marketing/list.php	579	Contact list CRUD.
app/admin/controller/marketing/mailserver.php	358	Mail server config CRUD.
app/admin/controller/marketing/message.php	143	Message templates.
app/admin/controller/marketing/newsletter.php	794	Newsletter CRUD + WYSIWYG body editor.
Admin models (750 lines)		
app/admin/model/marketing/bounce.php	194	Bounce model (vestigial, targets non-existent schema).
app/admin/model/marketing/campaign.php	230	Campaign model with on('save') hook.
app/admin/model/marketing/contact.php	113	Contact model.
app/admin/model/marketing/list.php	133	Contact list model (type bug on name).
app/admin/model/marketing/newsletter.php	80	Newsletter model (schema mismatch: \$table='post').
Storefront (278 lines)		
app/shop/controller/marketing/campaign.php	115	Tracking pixel + link redirect + link2Campaign.
app/shop/model/marketing/campaign.php	118	trackEmail, trackLink, getLink.
app/shop/model/marketing/contact.php	24	Stub.
app/shop/model/marketing/newsletter.php	21	Stub (same schema mismatch).
Cron (1,493 lines)		
system/cron/cron.php	245	Cron dispatcher (re-read).
system/cron/api/send.php	511	The campaign sender (most important file).
system/cron/api/bounce.php	167	Vestigial Interspire SendStudio copy.
system/cron/api/birthday.php	97	Birthday email sender.
system/cron/api/promoter.php	167	Promoter (referral) email sender.
system/cron/api/seller.php	92	Stub.
system/cron/api/update.php	214	Stub.
Email library (9,102 lines)		
system/library/email/mailer.php	2,289	PHPMailer 5.0 (renamed Mailer).
system/library/email/email.php	2,254	Base email class.
system/library/email/spam_rules.php	1,177	Spam rules.
system/library/email/vcard.php	1,290	vCard generator.
system/library/email/smtp.php	814	SMTP client.
system/library/email/pop3.php	406	POP3 client (for bounce).
system/library/email/template.php	130	Email template engine.
system/library/email/utf8.php	128	UTF-8 helper.
system/library/email/newsletter.php	53	Newsletter helper.
system/library/ntsmailer.php	164	NecoMailer wrapper.
system/library/mail.php	397	Legacy Mail class.
Admin API v1.0.0 (874 lines)		
app/admin/controller/api/v1.0.0/campaigns.php	154	Campaign CRUD API.
app/admin/controller/api/v1.0.0/campaigns_data.php	42	Field whitelist.
app/admin/controller/api/v1.0.0/contacts.php	151	Contact CRUD API.
app/admin/controller/api/v1.0.0/contacts_data.php	32	Field whitelist.
app/admin/controller/api/v1.0.0/contact_lists.php	149	Contact list CRUD API.
app/admin/controller/api/v1.0.0/contact_lists_data.php	31	Field whitelist.
app/admin/controller/api/v1.0.0/mailservers.php	102	Mailserver CRUD API.
app/admin/controller/api/v1.0.0/mailservers_data.php	34	Field whitelist.
app/admin/controller/api/v1.0.0/newsletters.php	147	Newsletter CRUD API.
app/admin/controller/api/v1.0.0/newsletters_data.php	32	Field whitelist.
Total	~17,000	

Table 7: Glossary of marketing-subsystem terms.

Term	Meaning
Newsletter	A reusable email body (HTML + text).
Campaign	A sending of a newsletter to a set of contacts.
Contact	A CRM record (optionally linked to a customer).
Contact list	A named collection of contacts.
campaign_contact	A per-recipient campaign membership row.
campaign_link	A trackable link registry row. The link column is the md5 nonce; redirect is the original href .
campaign_stat	An email-open event (tracking pixel).
campaign_link_stat	A link-click event.
task_queue	The per-recipient send queue. One row per recipient per campaign.
task_exec	The cron's lock table. One row per active job type.
Mailer	The PHPMailer 5.0 class (renamed). The SMTP workhorse.
NecoWidget	(From v3.) Not used in marketing.
Tracking pixel	A 1x1 JPEG appended to every email. Loading it triggers trace .
Link rewriting	The process of replacing <a href> in the newsletter HTML with trackable marketing/campaign/link URLs.
Double link rewriting	The admin commit phase and the cron send phase both rewrite links and insert campaign_link rows. The commit-phase rows are dead data if the per-recipient cache hits.
link2Campaign	A hidden follow-up-campaign trigger, fired by ?sendCampaign=<id> on the tracking endpoints.
Spam rules	A content-scoring system applied at compose time. Returns a numeric score + broken rules.
50-email throttle	The cron sender's hard-coded limit of 50 emails per pass, with a 15-minute defer.